Evaluating machine-learning learners with item response theory: a walkthrough

Eliot R. Weinstein · eliotw@uchicago.edu · erweinstein.com
November 2026

This walkthrough takes you through the code in this repository in the order you would run it. It shows what each step does, what its output looks like, and how to interpret it. The idea comes from treating a comparison of learners like a test:

- the learners are the examinees, and a bank of analysis tasks with known right answers is the test;
- item response theory (IRT) then puts the learners and the tasks on a common scale;
- because learners and tasks play symmetric roles, the model can also be "flipped", with the tasks as respondents and the learners as items.

The models follow Bock and Gibbons (2021, *Item Response Theory*), chiefly sections 11.5 (multilevel IRT) and 13.4 (IRT in machine learning), chapter 8 (adaptive testing) and the bifactor model of Gibbons and Hedeker (1992). The choices are mine, not a reproduction of anyone else's method.

Contents

1. What you need
2. The task bank
3. The learners
4. Running and scoring
5. Learners as examinees: a multilevel Rasch model
6. Flipped: learners as items
7. Choosing tasks adaptively
8. Adding a learner of your own
9. Cautions
10. References

1. What you need

R (4.3 or later) with glmnet, mgcv, FNN, rpart, ranger, xgboost (3.x), e1071, nnet, MASS, quadprog, lightgbm, dbarts, AER, ISLR2, mlbench, glmmTMB, mirt, statmod, ggplot2 and ggrepel. On R 4.3, mirt needs Deriv 4.1.6 from the CRAN archive. **Python** (optional, for CatBoost, RealMLP and TabPFN) with catboost, pytabkit, tabpfn 2.0.9 and a CPU build of torch, called from R through reticulate (1.47 or later, for NumPy 2). TabPFN's weights download from Hugging Face the first time it runs.

The MEPS tasks use `meps_ml.csv` from the MEPS walkthrough (using-meps-data-for-simulations, `ml_00_extract`). Put it at `../meps_for_simulation/data/meps_ml.csv`, or name it in the environment variable `MEPS_ML_CSV`. It is not in this repository, because AHRQ's data use agreement does not allow MEPS files, or row-level data made from them, to be redistributed. Without it the MEPS instances stop with an error and are skipped; everything else runs.

Run everything from the repository's top folder.

2. The task bank (`R/01_tasks.R`)

A **condition** is a kind of task, such as "predict a count outcome with mild overdispersion from 1,000 training rows". An **instance** is one draw of it: one simulated data set, or one random split of a real one. A **task-item** is an instance scored on one criterion. The bank has 95 conditions, 301 instances and 469 task-items in 13 families:

Family	What it tests	Task-items
Linear, nonlinear signal	simulated outcomes, 10–40 predictors, n = 250–3,000	45
Spending, yes/no, counts	a lump at zero and a long tail; prevalence 5–35%; overdispersion	156
Sparse, shift, informative missingness	100 predictors and n = 150; test data outside the training range; a predictor missing more often when the outcome is high	39
Importance	does permutation importance put the three real predictors on top?	12
Plasmode	MEPS adults' real predictors, outcomes from a known model fitted to MEPS	30
Real data	MEPS Panel 27, NMES, and seven teaching data sets	141
Prognostic adjustment	a trial analysed with a learned prognostic score (PROCOVA)	16
Treatment effect	cross-fitted AIPW with the learner for both nuisance functions	30

Each learner's answer to a task-item is 1 if it met the criterion and 0 if not. The continuous score behind it is kept too. The criteria are:

- **Accuracy.** For simulated and plasmode tasks the truth E[y | x] is known, so accuracy means a *signal* R^2 of at least 0.8:

$$R^2_{\text{signal}} = 1 - \frac{\left(\,\widehat{y} - \bar{E}\left[\,y \mid x\,\right]\,\right)^2}{\text{var}\,\left(\,E\left[\,y \mid x\,\right]\,\right)},$$

or, for yes/no outcomes, an AUC whose excess over 0.5 is at least 90% of the truth's. For real data there is no truth, so accuracy means coming within 0.01 of the best learner's R^2, AUC or explained deviance on the same split (0.02 for spending in dollars).

- **Calibration**: a calibration slope in [0.8, 1.25] with the mean prediction close to the training mean.
- **Efficiency**: for prognostic adjustment, a standard error within 5% of the one the true prognostic score would give.
- **Coverage**: for treatment effects, a 95% interval that contains the true effect.
- **Importance**: whether the top three predictors are the three real ones.

These thresholds are choices; `FLAGS.md` (item 1) says so.

3. The learners (`R/00_learners.R`, `py/learners_py.py`)

Every learner has the same interface, `fit_predict(name, type, xtr, ytr, xte)`, where `type` is `"gaussian"`, `"binary"`, `"count"` or `"cost"`. There are 19:

Group	Learners
Linear	OLS / logistic / Poisson regression; a loss-matched GLM (quasi-Poisson for spending, negative binomial for counts); lasso, ridge and elastic net (glmnet); a GAM (mgcv)
Local	k nearest neighbours; a radial support vector machine; a one-layer neural network; RealMLP (a modern multilayer perceptron, run in Python)
Trees	a pruned tree; a random forest; xgboost, LightGBM and CatBoost, each with defaults and tuned; BART
Other	a stack (non-negative weights on five learners' out-of-fold predictions); TabPFN v2 (a pretrained transformer that predicts in context)

The first 14 come from the MEPS walkthrough. The last five were added from recent benchmarks of learners for tabular data (TabArena, Erickson et al. 2025; TabPFN, Hollmann et al. 2025) and from causal inference (BART). Several are deliberately ordinary rather than best-tuned, as in many applied comparisons, so the results describe these configurations, not the algorithms in general. RealMLP and TabPFN are slow on a CPU (about 45–95 seconds

per fit at 1,000 training rows). To keep the run to a night, they answer only the first draw of each prediction task, so their other responses are missing by design, and TabPFN uses one ensemble member instead of its default four to eight. Item response models handle responses that are missing by design without trouble, but these two learners' estimates are less precise.

Tuned LightGBM and CatBoost were added on 13 November, tuned the way xgb_tuned is (not by a search): learning rate 0.05, small trees, and the number of rounds by 5-fold cross-validated early stopping. The tuned CatBoost also starts boosting from the training mean, which CatBoost's defaults do not do for log loss, Poisson or Tweedie (`FLAGS.md` items 20 and 22).

4. Running and scoring (`R/02_run.R`, `R/02b_combine.R`)

```
source("R/02_run.R")        # every learner on every instance; saves
output/instances/c<cond>_r<rep>.rds
source("R/02b_combine.R")   # output/responses.csv
```

The original 14 learners take about 50 minutes on two cores. The run is resumable: an interrupted run picks up where it stopped. Environment variables run part of it:

- `IRT_FAMILIES` chooses the families;
- `IRT_REPS` sets the number of draws per condition;
- `IRT_LEARNERS` chooses the learners;
- `IRT_INSTANCE_DIR` sets the output folder.

`run_v19.sh` runs, in chunks, the version that adds the five new learners (`IRT_PLAN=v19`). It took about 20 hours on two cores, including reruns after the seed fix below, mostly for TabPFN at 2,000–3,000 training rows (up to 20 minutes per fit, and about 3.1 GB of memory for TabPFN alone). To fit in a session with a 2-hour limit per job and 5.8 GB of memory:

- `run_v19_queue.sh TAG SPEC...` runs chunks in order and stops after 112 minutes; rerun it and it skips the instances already saved. A SPEC is `families|replicates|cores|n_min|n_max`.
- `run_v19_fix.sh TAG` runs two queues side by side: the large first draws one at a time on one core, and the fast learners on the other.

A seed bug, fixed. `load_meps()` used to set the seed to 1 the first time it read MEPS, which overwrote the instance's own seed, so in the original run the three draws of each MEPS-based condition were the same data set (`FLAGS.md` item 19). The seed is now saved and restored around it. Two scripts check and repair the 19-learner run:

- `R/02d_check_meps_splits.R` regenerates every stored MEPS-based instance and checks it against its own split (`output/meps_split_check.csv`);
- `R/02e_add_original_plasmode.R` adds the original 14 learners to plasmode instances that were rerun with the new five only.

`R/02c_merge_v19.R` then merges the 19-learner run with the original one into `output/instances_19`. On the real-data and plasmode instances every learner was rerun, so it takes the rerun; on the simulated ones it adds the new learners' answers to the original 14's. `R/02b_combine.R`, with `IRT_INSTANCE_DIR` and `IRT_RESPONSES` set, writes `output/responses_19.csv`.

`PLAN=v21 ./run_v19_queue.sh TAG "||1|0|Inf"` adds the tuned LightGBM and CatBoost (about 20 minutes on one core). Real-data accuracy is judged against the best learner on the split, and only the difference is stored, so on the real-data instances the 17 learners run on every instance are rerun to rebuild that reference. They reproduce their stored scores exactly, and `R/02f_merge_v21.R` checks this as it merges. The reference stays the best of those 17, so adding the tuned learners changes no other learner's answers. The results with 21 learners are in `output/responses_21.csv` and `output/v21/`.

`output/responses.csv` has one row per learner and task-item:

```
cond, inst, rep, criterion, learner, score, pass, secs, family, source, type, n, detail
1, c001_r1, 1, acc, glm, 0.965, 1, 2.3, lin, sim, gaussian, 250, 10
```

Before any IRT, look at the proportion of criteria each learner met (`output/irt_prop_correct.csv`). It is the classical test score, and the ability estimates below put the learners in almost the same order (Spearman 0.96 for the original 14, 0.93 for all 19). Ordering learners is not where IRT adds most.

5. Learners as examinees: a multilevel Rasch model (`R/03_irt.R`, part A)

Let $Y_{li} = 1$ if learner l meets the criterion on task-item i. Task-items are nested in instances, instances in conditions, and conditions in families, so the task side of the model is multilevel.

Level 1 (learner l, task-item i):

$$\text{logit}\ P(\ Y_{li} = 1\ )\ = \theta_l + \eta_i + w_{lf(\,i\,)}$$

Level 2 (task-item i in instance j): $\eta_i = \pi_{0j} + c_{\text{crit}\,(\,i\,)} + r_i$, with $r_i \sim N(\,0, \tau_r\,)$

Level 3 (instance j in condition k): $\pi_{0j} = \beta_{0k} + u_j$, with $u_j \sim N\big(\,0, \tau_u\,\big)$

Level 4 (condition k in family f): $\beta_{0k} = \gamma_f + v_k$, with $v_k \sim N\big(\,0, \tau_v\,\big)$ and $\gamma_f \sim N\Big(\,0, \tau_\gamma\,\Big)$

Combined:

$$\text{logit}\ P(\ Y_{li} = 1\ )\ = \theta_l + c_{\text{crit}\,(\,i\,)} + \gamma_f + v_k + u_j + r_i + w_{lf}$$

Here:

- θ_l is learner l's ability, a fixed effect because there are so few learners;
- η_i is the task-item's easiness, and c_{crit} a shift for each criterion;

- $w_{lf} \sim N\left(0, \tau_w\right)$ is a learner-by-family interaction. Without it this is a Rasch model, and $\tau_w > 0$ means a learner's ability depends on the kind of task.

In R, with glmmTMB (lme4 failed on these data; `FLAGS.md` item 5):

```
A1 <- glmmTMB(pass ~ 0 + learner + criterion + (1 | task_family) + (1 | cond) + (1 | inst)
+
                (1 | item) + (1 | learner:task_family), data = R, family = binomial)
```

What to look at.

- `output/irt_A_ability.csv`: abilities, centred at zero, with SEs. The SEs come from the model with the interaction and are of the right size (section 7 explains why that matters).
- `output/irt_A_profile.csv` and `fig_learner_family_profile.png`: each learner's deviation by family. This shows *where* a learner is strong, which a single score hides.

What you should see (all 19 learners, from `output/v19/`; ability in logits, centred at zero, and the proportion of criteria met):

Learner	Ability (SE)	Met	Learner	Ability (SE)	Met
stack	3.46 (0.68)	0.80	glm	0.50 (0.65)	0.56
gam	1.81 (0.65)	0.66	realmlp	−0.51 (0.73)	0.38
bart	1.75 (0.65)	0.68	nnet	−1.19 (0.66)	0.33
tabpfn	1.67 (0.73)	0.62	catboost	−1.68 (0.67)	0.28
enet	1.62 (0.65)	0.68	svm	−1.92 (0.66)	0.29
lasso	1.49 (0.65)	0.70	cart	−2.08 (0.66)	0.33
glm_matched	1.39 (0.97)	0.62	lightgbm	−2.82 (0.68)	0.24
xgb_tuned	1.33 (0.64)	0.65	knn	−2.82 (0.67)	0.22
ridge	1.21 (0.65)	0.64	xgb_default	−3.94 (0.72)	0.20
rf	0.74 (0.64)	0.54			

The SEs are large, about 0.65 logits, so learners close together in the table are not reliably ordered; the clear contrast is between the upper group (the stack down to glm) and the six from CatBoost down. LightGBM and CatBoost run at their package defaults and rank with the untuned xgboost, not the tuned one (`FLAGS.md` item 20). Tuned, they join the upper group: in the 21-learner fit (`output/v21/`) the tuned CatBoost's ability is 1.36 and the tuned LightGBM's 0.97, beside tuned xgboost's 1.21. glm_matched answers only spending and count tasks, hence its larger SE.

6. Flipped: learners as items (`R/03_irt.R`, part B)

Now the task-items are the respondents and the learners the items. With learners as items, each learner gets a **discrimination** (how sharply its success rises with task easiness) as well as a difficulty, and the multidimensional and bifactor models describe which learners succeed and fail together.

One factor (2PL):

$$\text{logit}\, P(\, Y_{il} = 1 \mid \theta_i \,) \;= a_l \,(\, \theta_i - b_l \,)\,, \qquad \theta_i \sim N(\, 0, 1 \,)$$

Here θ_i is the task-item's easiness. b_l is the easiness a task needs for learner l to have an even chance, and a_l is learner l's discrimination. A learner with a large a_l is predictable from how easy a task is; one with a small a_l succeeds or fails for reasons the common dimension does not capture.

Bifactor:

$$\text{logit}\, P(\, Y_{il} = 1 \mid \mathbf{\theta}_i \,) \;= a_{l0}\theta_{i0} + a_{l,\,s\,(\,l\,)}\,\theta_{i,\,s\,(\,l\,)} + d_l$$

This has a general factor plus one group factor for the learner's group $s\,(\,l\,)$ (linear, trees, local). The stack and TabPFN load on the general factor only.

In R, with mirt (`Y` is a task-items × learners matrix of 0, 1 and NA):

```
B1 <- mirt(Y, 1, itemtype = "2PL")                # one factor
BB <- bfactor(Y, spec)                             # spec: each learner's group (NA =
general only)
coef(B1, IRTpars = TRUE, simplify = TRUE)$items    # a and b for each learner
M2(B1, na.rm = TRUE)                               # limited-information fit
```

Two levels. A random effect for the task condition splits each learner's discrimination in two (Bock and Gibbons's eq. 11.60):

Level 1 (learner l, task-item i in condition k): $\text{logit}\, P(\, Y_{il} = 1 \,) \;= d_l + c_{\text{crit}\,(\,i\,)} + a_l^B \zeta_k + a_l^W \zeta_i$

Level 2 (task-item i): $\zeta_i \sim N(\, 0, 1 \,)$

Level 3 (condition k): $\zeta_k \sim N(\, 0, 1 \,)$

Combined: $\text{logit}\, P(\, Y_{il} = 1 \,) \;= d_l + c_{\text{crit}\,(\,i\,)} + a_l^B \zeta_k + a_l^W \zeta_i$

Here a_l^B is learner l's discrimination between kinds of task and a_l^W its discrimination within a kind. It is fitted with glmmTMB's reduced-rank terms (mirt's `mixedmirt()` stopped on these data):

```
ML2 <- glmmTMB(pass ~ 0 + learner + criterion + rr(0 + learner | condf, d = 1) +
                 rr(0 + learner | itemf, d = 1), data = R, family = binomial)
```

What to look at.

- `irt_B_items_2pl.csv` and `fig_learner_curves.png`: the learner characteristic curves.

- `irt_B_bifactor.csv`: whether a group factor is real, meaning learners in a group share successes beyond the general factor.
- `irt_B_multilevel.csv` and `fig_two_level_discrimination.png`: which learners are sensitive to the kind of task and which to the particular draw.
- `irt_B_dif.csv`: differential functioning between simulated and real-data tasks (Bock and Gibbons ch. 9).

Near-duplicate learners (lasso, ridge, elastic net) agree on 89–94% of task-items and so get very large slopes (11–23; apart from TabPFN, the others' run from 0.8 to 5.5). `irt_B_items_2pl_without_ridge_enet.csv` is the refit without two of them. TabPFN's slope is near zero (0.13, SE 0.20): whether it succeeds does not follow task easiness the way the other learners' successes do. It passes accuracy criteria often but underpredicts the mean on spending and counts (`FLAGS.md` item 21).

7. Choosing tasks adaptively (`R/05_cat.R`, `R/06_bifactor_cat.R`)

A new learner should not need the whole bank if each next task-item can be chosen to be the most informative one. Both scripts test this by leaving one learner out:

- calibrate the bank on the other learners;
- test the left-out learner adaptively, using its actual answers;
- compare its ability after k items with what its answers to the whole bank give.

Rasch selection does not work here (`05_cat.R`). Choosing the item with the most Rasch information was no more accurate than choosing at random, and the posterior SDs were too small. The reason is **local dependence**. With learner-by-family and learner-by-condition effects in the model, their SDs are 2.2 and 1.7 logits, larger than the spread of the abilities (SD 1.4 in the model without them): a learner's answers within one condition go together beyond its overall ability.

Bifactor selection (`06_bifactor_cat.R`) handles this, as Gibbons's adaptive tests handle subdomains (Gibbons et al. 2020; Tseng et al. 2026). Each item measures the primary ability and a specific dimension, and the primary score integrates the specific dimensions out (Gibbons et al. 2007, eqs. 17–18). A handful of calibration learners cannot estimate loadings, so every loading is fixed at 1 (a Rasch testlet version of the bifactor model), with two nested specific dimensions:

Level 1 (learner l, task-item i in condition c of family f): $\text{logit}\, P\left(Y_{li}=1\right) = \theta_l + u_{lf} + w_{lc} + e_i$

Level 2 (learner l): $\theta_l \sim N\left(\mu, s^2\right)$; $u_{lf} \sim N\left(0, \sigma_f^2\right)$ for each family; $w_{lc} \sim N\left(0, \sigma_c^2\right)$ for each condition

The likelihood for θ is a product over families of nested integrals (over u_f, and over each w_c within f), computed by Gauss–Hermite quadrature on a grid for θ. The next item is the one that most reduces the expected posterior variance of θ, taken with probability 0.8, and the second best with 0.2 (CAT-Autism's exposure rule).

What you should see (all 19 learners; mean distance from the full-bank ability, in logits):

Items given	Bifactor selection	Rasch selection	Random
10	0.72	0.92	0.93
20	0.53	0.77	0.70
40	0.35	0.66	0.56
80	0.20	0.38	0.35

The errors are all measured against the full-bank ability under the bifactor model (`FLAGS.md` item 13's Rasch figures use the Rasch model's full bank). The target was a posterior SD within 10% of the full bank's. Within 80 items the bifactor rule reached it in 28 of 57 runs, Rasch selection in 21 of 57 and random selection in 62 of 190. In the runs that reached it, they needed a median of 34, 49 and 56.5 items. So most runs of every rule did not reach the target, and those medians understate the items it takes. But under the bifactor model even the full bank pins a learner down only to a posterior SD of 0.39–1.01 logits. The bank is less informative than its 469 items suggest.

8. Adding a learner of your own

1. Add a branch to `fit_predict()` in `R/00_learners.R` (or to `py/learners_py.py` for a Python learner, with its name in `PY_LEARNERS`). It must return predictions for `xte`: probabilities for `"binary"`, means otherwise. Return `NULL` for an outcome type it does not handle; that response is then missing by design.
2. Add its name at the end of `LEARNERS`, so the existing learners keep their seeds and predictions.
3. Run it alone on every instance: `IRT_LEARNERS=<name> IRT_INSTANCE_DIR=output/instances_new Rscript R/02_run.R`. Real-data accuracy is judged against the best learner, so rerun every learner on the real-data instances, as `IRT_PLAN=v19` does.
4. Merge, combine and refit as in `R/02c_merge_v19.R`, then `03_irt.R` and `06_bifactor_cat.R` with `IRT_RESPONSES` and `IRT_OUT` set.

Or, to place a new learner on the existing scale without refitting, run it on the bank and score it adaptively with the calibration from the others (section 7).

9. Cautions

- **Few examinees.** IRT models are usually calibrated on hundreds or thousands of examinees (801 for the K-CAT, over 10,000 for CAT-Autism); here there are 19. Loadings and slopes are poorly determined, and absolute fit is poor (M2 RMSEA about 0.2).
- **The criteria are thresholds**, and a learner's configuration, not its algorithm, is what is scored. `R/08_threshold_sensitivity.R` recomputes pass/fail under stricter and more

lenient thresholds and refits the model: the stack stays first and the last seven stay last, but the order within the upper group changes (Spearman 0.86–1.00 with the baseline; `FLAGS.md` item 1).

- **Relative criteria.** Real-data accuracy is relative to the best learner on the split, so adding a learner to the reference can change other learners' answers (the tuned learners of 13 November were kept out of it for that reason).
- **Local dependence is large.** Treating the 469 task-items as independent overstates the precision of any ability estimate; model the nesting.
- **Location-scale models.** A mixed-effects location-scale model of draw-to-draw variability (Hedeker, Mermelstein and Demirtas 2008; fitted with Wu and Hedeker's 2026 melsvmp) was degenerate on these data (`R/04*`, `FLAGS.md` item 16).
- **TabPFN's low means on spending and counts** (predicted mean a median 0.77 of the training mean on the 14 instances with up to 1,000 training rows). `R/07_tabpfn_check.R` refits it in four variants. The cause is not the bounded range of its predicted distribution, nor the target transform its preprocessing draws. Fitting on log(1 + y) raises the median to 0.91 and meets the calibration criterion on 7 instances instead of 4, but it overshoots on one and lowers two, and most of the shortfall on counts remains. The default 8-member ensemble is still being checked (`FLAGS.md` item 21).

10. References

- Bock, R. D. and R. D. Gibbons (2021). *Item Response Theory*. Wiley.
- Brooks, M. E., et al. (2017). glmmTMB balances speed and flexibility among packages for zero-inflated generalized linear mixed modeling. *The R Journal* 9(2), 378–400.
- Chalmers, R. P. (2012). mirt: a multidimensional item response theory package for the R environment. *Journal of Statistical Software* 48(6), 1–29.
- Erickson, N., L. Purucker, A. Tschalzev, D. Holzmüller, P. M. Desai, D. Salinas and F. Hutter (2025). TabArena: a living benchmark for machine learning on tabular data. *NeurIPS 2025 Datasets and Benchmarks Track*; arXiv:2506.16791.
- Gibbons, R. D. and D. R. Hedeker (1992). Full-information item bi-factor analysis. *Psychometrika* 57(3), 423–436.
- Gibbons, R. D., R. D. Bock, D. Hedeker, D. J. Weiss, E. Segawa, D. K. Bhaumik, D. J. Kupfer, E. Frank, V. J. Grochocinski and A. Stover (2007). Full-information item bifactor analysis of graded response data. *Applied Psychological Measurement* 31(1), 4–19.
- Gibbons, R. D., D. J. Kupfer, E. Frank, B. B. Lahey, B. A. George-Milford, C. L. Biernesser, G. Porta, T. L. Moore, J. B. Kim and D. A. Brent (2020). Computerized adaptive tests for rapid and accurate assessment of psychopathology dimensions in youth. *Journal of the American Academy of Child & Adolescent Psychiatry* 59(11), 1264–1273.
- Hedeker, D., R. J. Mermelstein and H. Demirtas (2008). An application of a mixed-effects location scale model for analysis of ecological momentary assessment (EMA) data. *Biometrics* 64(2), 627–634.

- Hollmann, N., S. Müller, L. Purucker, A. Krishnakumar, M. Körfer, S. B. Hoo, R. T. Schirrmeister and F. Hutter (2025). Accurate predictions on small data with a tabular foundation model. *Nature* 637, 319–326.
- Martínez-Plumed, F., R. B. C. Prudêncio, A. Martínez-Usó and J. Hernández-Orallo (2016). Making sense of item response theory in machine learning. *Frontiers in Artificial Intelligence and Applications* 285, 1140–1148.
- Tseng, A., A. Yang, L. Mandocescu, R. D. Gibbons and S. Jacob (2026). Computerized adaptive tests for rapid and accurate assessment of autism. *JAMA Network Open* 9(7), e2622227.
- Wu, B. P.-H. and D. Hedeker (2026). Fast estimation and valid statistical inference for mixed-effect location-scale models using variational inference. *Statistics in Medicine* 45, e70640.